DT081A — Laboratory 2 Report

Student: Jakob Obid

2026-11-05

1 Introduction

This lab studies signals in the time and frequency domains. The tasks include windowing, LTI systems, digital filters and voice analysis.

2 Experimental Setup

I used Python with NumPy, SciPy and Matplotlib. For the first task, I sampled three cosine waves at 250 Hz for 0.6 seconds. Their frequencies were 50, 55 and 120 Hz. I compared rectangular, Hann and Hamming windows using the same signal. I also tested FIR and IIR systems using unit impulses. I also calculated the amplitudes and phases of the three original cosine waves. I also passed a 50 Hz cosine through System 2 and compared its input and output spectra. I designed Butterworth and Chebyshev filters and compared different cutoff frequencies, filter types and orders. I also analyzed my voice recording using different segment lengths.

3 Results and Analysis

3.1 Time-Domain Analysis

The signal contains three cosine waves. The FFT shows peaks at 50, 55 and 120 Hz (Figure 1).

Q1. Why does the amplitude spectrum contain negative frequencies?

Answer: The signal contains real cosine waves, so its FFT has matching positive and negative frequencies. The negative side is a mirror of the positive side. It does not mean extra frequencies were added to the signal (Figure 1).

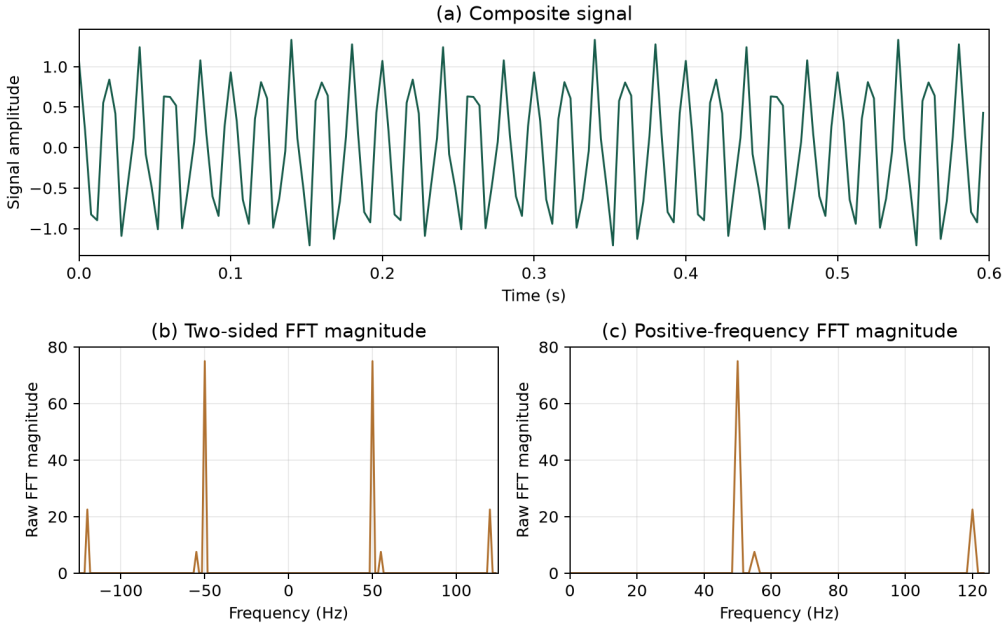


Figure 1: Composite signal, two-sided FFT magnitude and positive-frequency FFT magnitude. FFT magnitudes are unnormalized.

3.2 Frequency Identification

Q1. Which of the windows (rectangular, Hann, and Hamming) is best able to separate frequencies close to each other? Why?

Answer: The rectangular window separated the 50 Hz and 55 Hz frequencies most clearly in this test. Both frequencies fall exactly on FFT points, so the peaks stay narrow. Hann and Hamming made the peaks wider, but they can reduce leakage when frequencies do not fall exactly on FFT points (Figure 2).

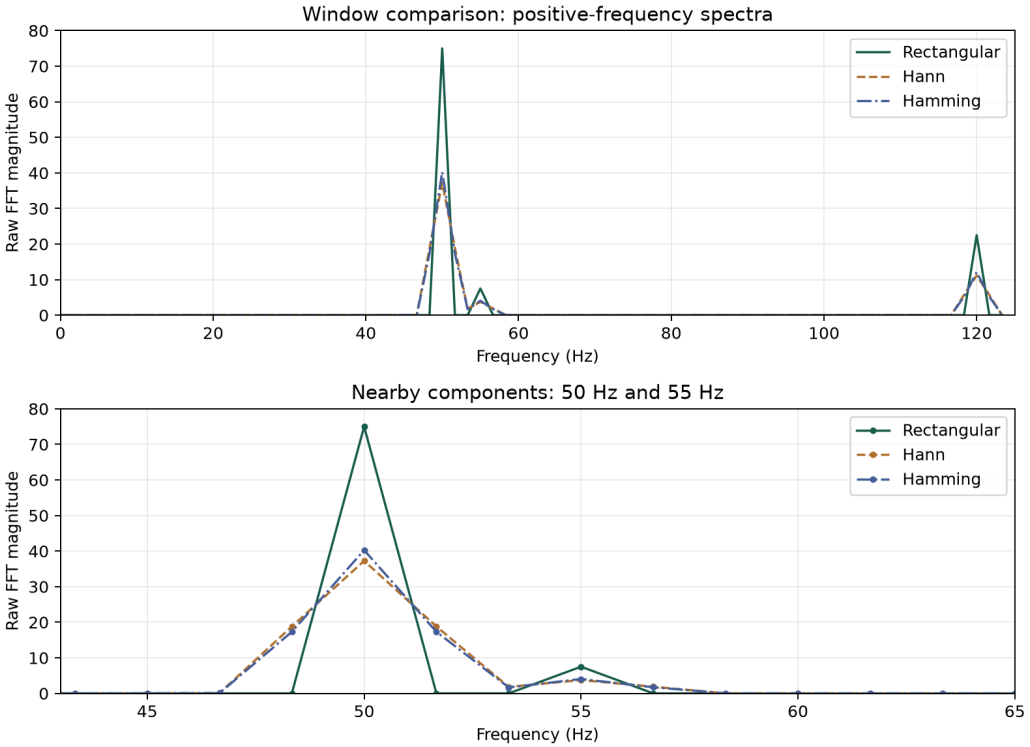


Figure 2: Rectangular, Hann and Hamming window spectra. The lower plot zooms in on the 50 Hz and 55 Hz components.

3.3 Linear Time-Invariant Systems

System 1

Q1. Does the convolution give the correct response?

Answer: Yes. I convolved the filter coefficients with a unit impulse. The first eight output samples matched the given coefficients, and the remaining samples were zero (Figure 3).

Q2. Is this a FIR or IIR system? How can you tell?

Answer: This is an FIR system because its impulse response ends after eight samples. There is no feedback from earlier output values.

System 2

Q1. Implement the system with a loop and plot the output.

Answer: I used a loop to calculate 100 output samples from a unit impulse. The first five values were 1, -0.3, 0.53, -0.423 and 0.4393 (Figure 3). I followed the supplied filter coefficients, which use the previous two output samples.

The printed difference equation repeats $y(n-1)$, but the given filter coefficients require $y(n-2)$ in the final term.

Q2. Is this a FIR or IIR system? How can you tell?

Answer: This is an IIR system because the output depends on earlier output values. Its impulse response continues instead of stopping after a fixed number of samples.

Q3. Do you get the same results in the two simulations?

Answer: Yes. The loop and `lfilter()` gave the same 100 output values. The maximum difference was zero (Figure 3).

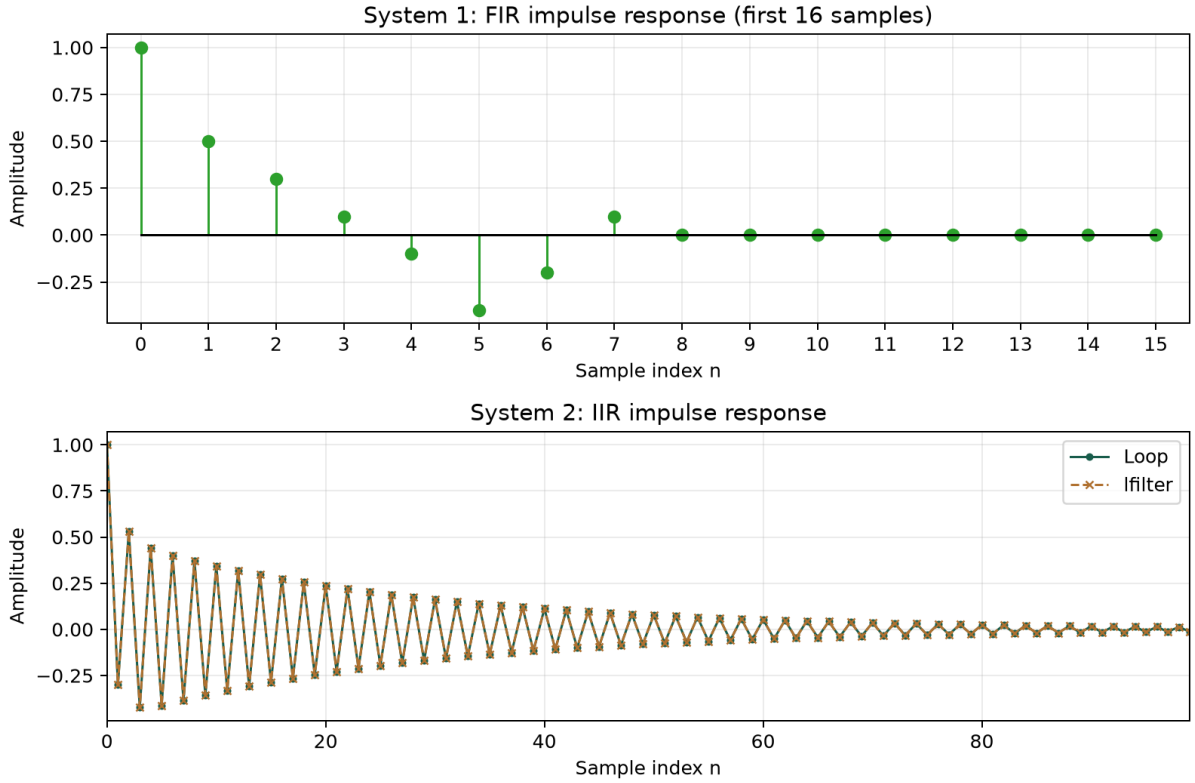


Figure 3: Impulse responses of the FIR and IIR systems. The loop and lfilter outputs agree for System 2.

3.4 Frequency-Domain Analysis

Q1. From these spectra, decide the frequencies, amplitudes, and phases of the sinusoids in the signal. Use the zoom-functionality in the graphs to see better. Do the amplitudes and phases agree with what you have defined in the code?

Answer: I found peaks at 50, 55 and 120 Hz. I divided the raw FFT magnitudes by 75 to get the original amplitudes. The amplitudes were 1, 0.1 and 0.3, and the phases were 0, 0.7854 and 1.5708 radians. All values matched the original signal (Figure 4).

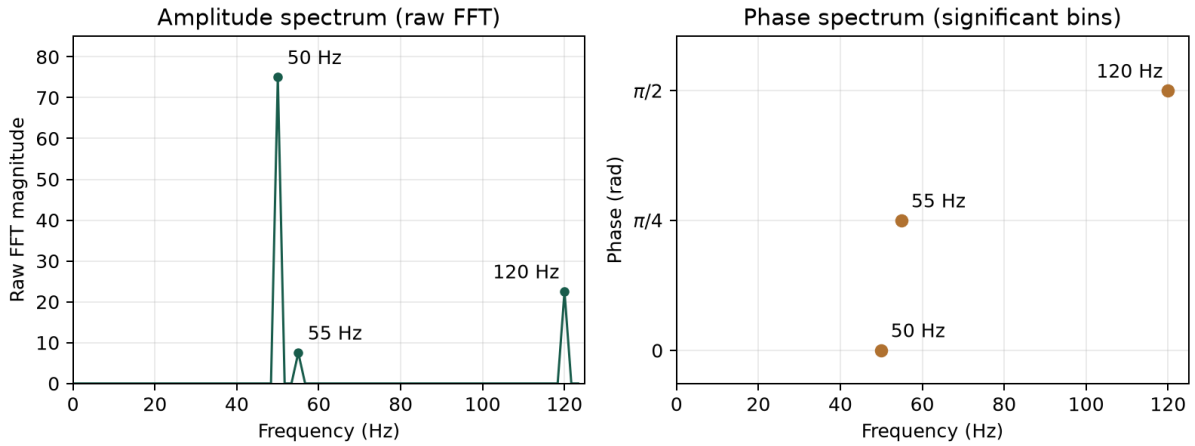


Figure 4: Amplitude and phase spectra of the three cosine waves.

3.5 System Frequency Response

Q1. Identify the amplitude and phase of both input and output signals by analysing them in the frequency domain as in Section 4.

Answer: I sent a 50 Hz cosine through System 2. The input amplitude was 1 and its phase was 0 rad. The output amplitude was 0.7481 and its phase was -0.0103 rad (Figure 5).

Q2. Analyse the system frequency response in the frequency domain. Verify the relation $Y(f) = H(f)X(f)$ for both amplitude and phase.

Answer: At 50 Hz, the system gain was 0.7454 with a phase shift of -0.0128 rad. This predicts an output amplitude of 0.7454 and a phase of -0.0128 rad. The measured values were 0.7481 and -0.0103 rad. They were close but not exactly equal because the recording was short and included the filter's start-up response (Figure 5).

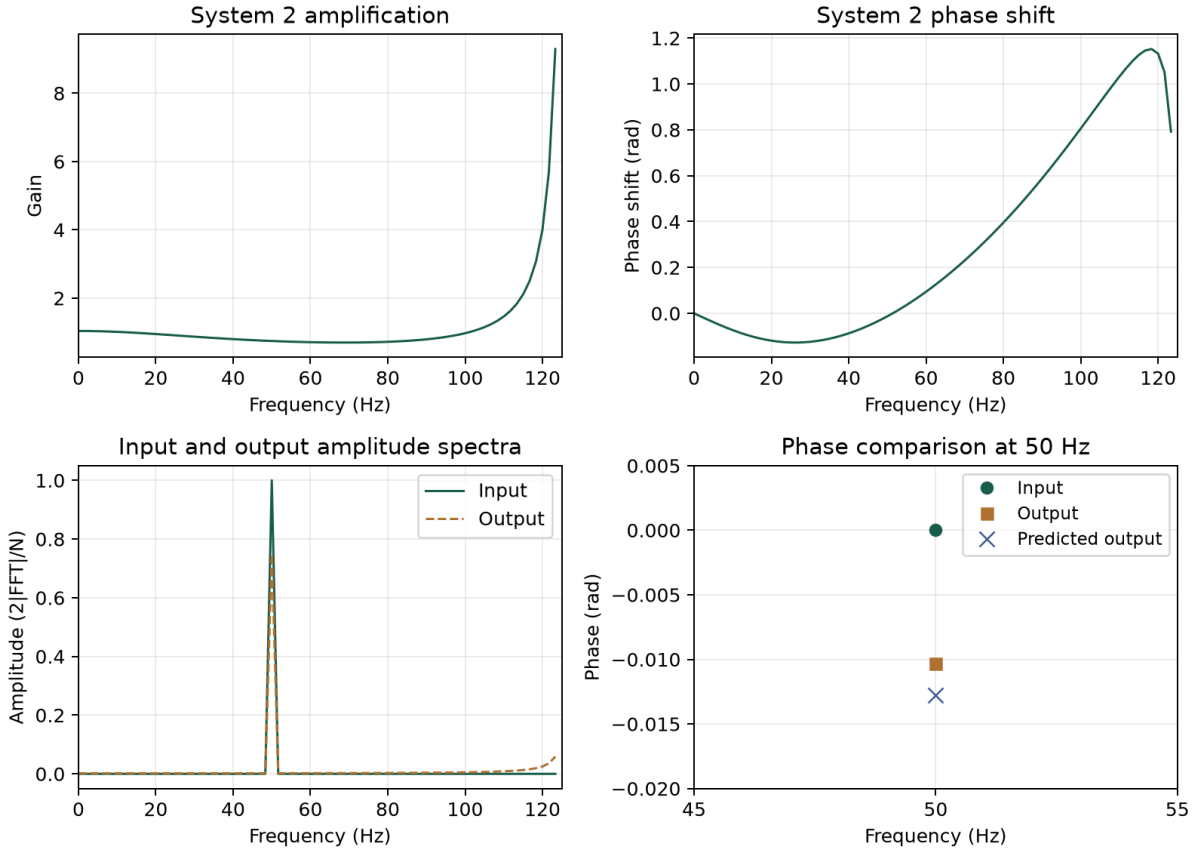


Figure 5: System 2 frequency response and the amplitude and phase of the 50 Hz input and output signals.

3.6 Filtering

Q1. How do Butterworth and Chebyshev type 1 differ?

Answer: Butterworth has a smooth passband, while Chebyshev Type I has ripple. Chebyshev also reduces frequencies more sharply after the cutoff. At 300 Hz, the low-pass gains were about -18.87 dB and -36.42 dB (Figure 6).

Q2. How do the filters differ in amplification and phase shift when changing LP/HP, f_c, order N?

Answer: Low-pass filters keep lower frequencies, while high-pass filters keep higher ones. Raising the cutoff from 200 to 400 Hz allowed more high frequencies through the low-pass filter. Increasing the order from 3 to 7 made the transition sharper. At 100 Hz, the phase also changed from about -1.03 to -2.28 radians (Figure 6).

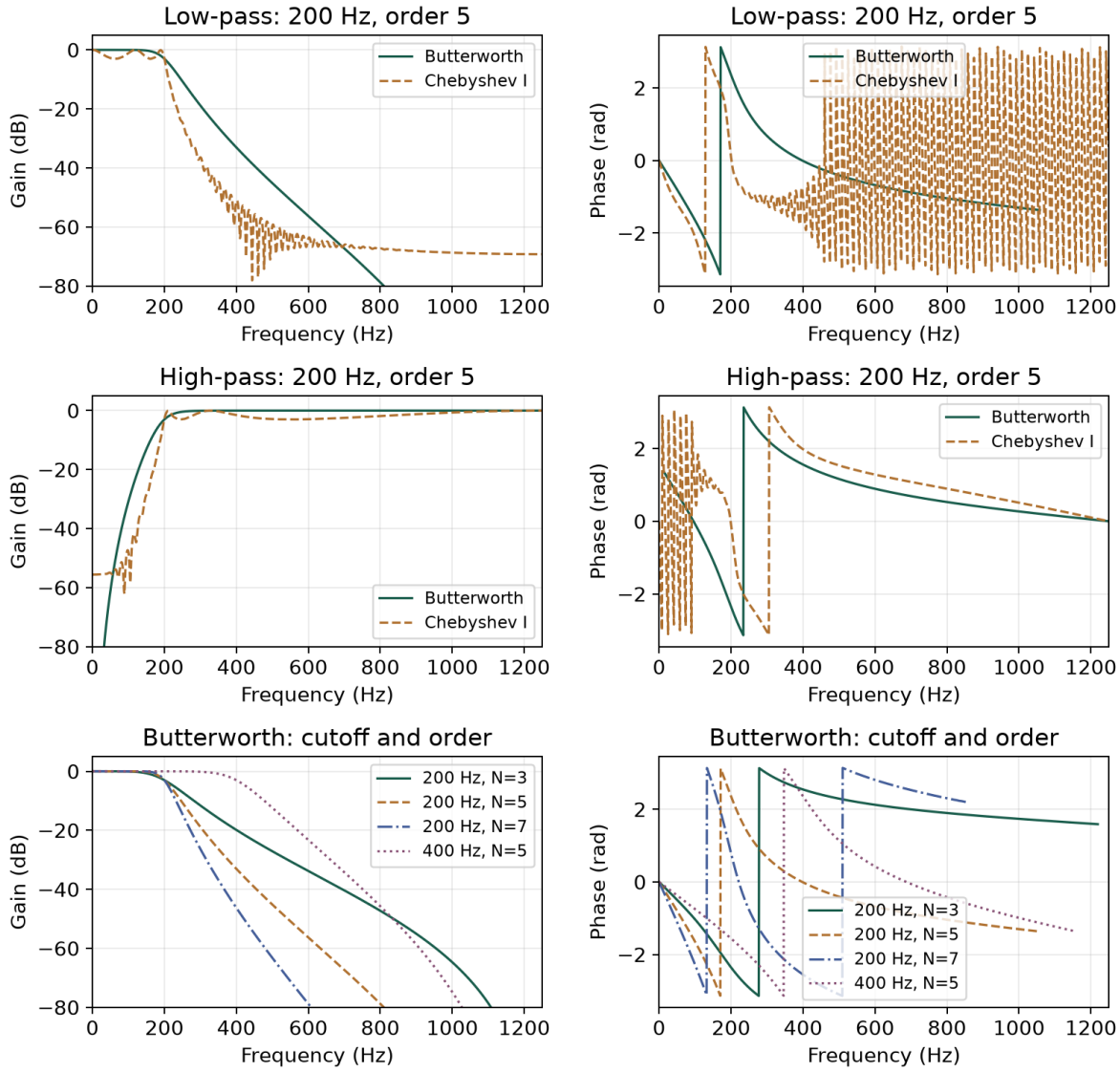


Figure 6: Butterworth and Chebyshev filter responses, with comparisons of filter type, cutoff frequency and order.

Q3. Write the Python code to design BP and BS filters.

Answer: I designed Butterworth band-pass and band-stop filters using 250 Hz and 500 Hz as cutoff frequencies. The band-pass mainly keeps frequencies between them. The band-stop reduces frequencies in that range.

```
band = [250, 500]
b_bp, a_bp = signal.butter(
    5, band, btype="bandpass", fs=2500)
b_bs, a_bs = signal.butter(
```

```
5, band, btype="bandstop", fs=2500)
```

3.7 Frequency-Time Domain Analysis

Q1. What is $X(t, f)$ called? What is $|X(t, f)|^2$ called?

Answer: $X(t, f)$ is the short-time Fourier transform (STFT). It shows how the frequencies change over time. Its squared magnitude gives the time-frequency power spectrum, which is shown as a spectrogram (Figure 7).

Q2. How does changing the length of the sub-sequences affect time and frequency resolution? Try out different lengths by changing k in the code.

Answer: I tested segment lengths of 512, 1024 and 2048 samples. Shorter segments showed when sounds changed more clearly, but had poorer frequency resolution. Longer segments gave better frequency resolution but less detail in time (Figure 7).

Segment length k	Time (s)	Frequency spacing (Hz)
512	0.010667	93.7500
1024	0.021333	46.8750
2048	0.042667	23.4375

Q3. Try to identify the letters of your name by looking at $|X(t, f)|^2$.

Answer: I examined the spectrogram of my spoken name. Different sound regions appeared at different times. I could see changes between parts of the name, but individual letters were difficult to identify from the spectrogram alone (Figure 7).

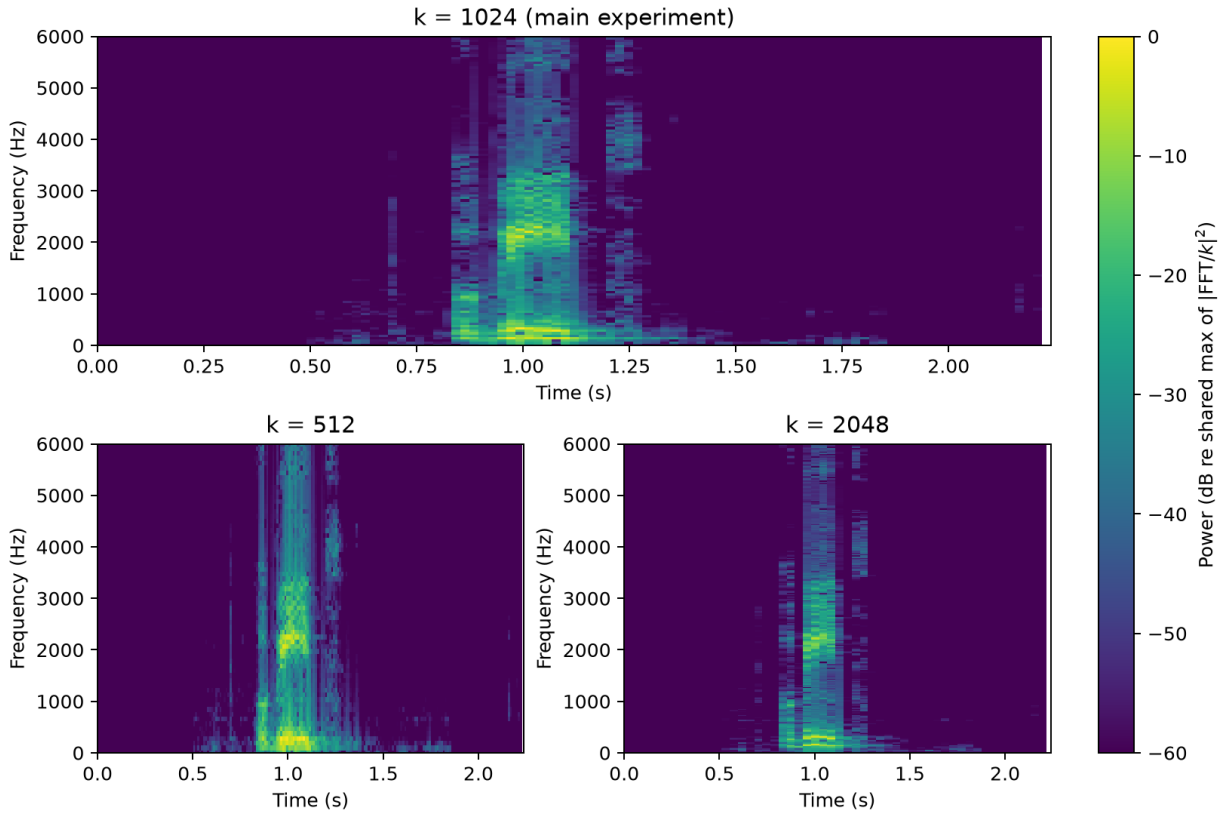


Figure 7: Time-frequency power spectrum of Obid's voice recording using segment lengths of 512, 1024 and 2048 samples.

4 Discussion

The rectangular window separated the 50 Hz and 55 Hz peaks best because both frequencies matched the FFT bins. Hann and Hamming made the peaks wider, but they can reduce leakage when frequencies fall between bins. The filter tests showed another difference. Chebyshev had a sharper cutoff than Butterworth, but it also had ripple in the passband.

For System 2, the measured output amplitude was 0.7481 instead of the predicted 0.7454. This small difference is consistent with the short recording and the filter's start-up response. In my voice experiment, shorter segments showed changes in time more clearly, while longer segments showed frequencies more precisely. I could not improve both types of resolution at the same time.

5 Conclusion

The first experiment showed how the FFT can identify the frequencies in a signal. The window comparison showed how different windows change the frequency spectrum. The two LTI experiments showed the difference between finite and recursive impulse responses. The frequency analysis confirmed the original amplitudes and phases. System 2 reduced the 50 Hz tone and caused a small phase shift. The filtering tests showed how filter type, cutoff frequency and order affect amplification and phase. The spectrogram showed how the frequencies in my voice changed over time.

Appendix: Code Snippets

Example 1 – System 2: IIR Recurrence

```
for n in range(len(impulse)):
    y_loop[n] = impulse[n]
    if n >= 1:
        y_loop[n] += 0.3 * impulse[n-1] - 0.6 * y_loop[n-1]
    if n >= 2:
        y_loop[n] += 0.35 * y_loop[n-2]
```

I used a loop to calculate each output sample. The system uses previous output values, which makes it an IIR system.

Example 2 – Voice FFT Analysis

```
k = 1024
n = len(x_voice) // k
segments = x_voice[:n*k].reshape(n, k)
X_voice = np.fft.rfft(segments, axis=1)
power = np.abs(X_voice)**2
```

I divided my voice recording into short segments and calculated the FFT of each one. The squared magnitude shows how the frequency content changes over time.